

Predicting the Second Gift — executive summary

Group 11 · GBUS 8496 · Malorie Black, Reid Jacobson, Thadeus Knospe, Rodolfo Perez-Cortes Manrique, Bakul Badwal · October 2026

Problem

On DonorsChoose, the classroom crowdfunding platform, 71% of donors across seventeen years gave exactly once. The second gift is where retention is won or lost. A development lead at a \$500K nonprofit with no data staff can personally follow up with only a fraction of last month's first-time donors, and today she picks them by hand from gift size. Our question: at a fixed monthly capacity, which first-time donors should get the call? One of us founded GoGood Technologies, whose customers are this stakeholder; weigh our conclusions with that in mind.

Approach

We used DonorsChoose Open Data 2002–2019 (ICPSR 37898): 11.4 million donations from 3.5 million donors. Each donor gets one label, whether they gave again within twelve months of their first gift. The 2017–18 cohorts were held out during model selection, because her question is about next year's donors. Every rule is scored at her capacity, the top 10% of each month's new donors, in dollars of subsequent giving identified per contact. The baseline is her own rule, rank by first-gift size.

One finding shaped everything after it: the file holds three populations. Organizations are 0.1% of donors but 62% of repeat dollars, mostly corporate matching programs. Teachers seed their own classrooms. Pooled, any rule looks brilliant by finding the corporations. We scored individual donors, 86% of people, the ones she actually calls.

Findings

A model beats her rule by a small, reliable margin. On the holdout, ranking by predicted second-gift value identifies **\$120 per contact**, against **\$116** for gift size and **\$21** for selecting everyone. The paired bootstrap gap is $+\$3.51 \pm 0.51$ (one standard error, 30 resamples). One call in ten reaches 58% of all subsequent giving. The model is two gradient-boosted estimators, for the chance of a second gift and its size, using first-gift attributes plus the project and school first funded. Gift size carries much of the signal; the full model adds a modest gain.

The result holds up, within limits. When the model says its top tenth of donors has a 27% chance of returning, 24.5% actually do. It beats gift size in both test years (\$139 per donor in 2017, \$103 in 2018). What it changes is who gets the call: it swaps out teacher-referred donors, friends and family recruited for one classroom who return 13% of the time, for donors who funded several classrooms at once, who return 23%. Its list is still mostly larger first gifts; donors under \$50 are half of all new donors but under 1% of its calls. Ranking purely by the chance of return reaches more people but fewer dollars (\$108 per call), so the right list depends on whether the goal is more donors or more dollars.

Recommendation

Use the model to build each month's call list, and prove it with a pilot. Rank new individual donors by expected value and call the top 10%, then run a randomized pilot against the gift-size list to measure what outreach actually adds before scaling. On the holdout, the model's list holds \$9.76M of next-year giving; after an assumed \$25 per call it leaves \$7.72M, versus \$7.44M for the gift-size list. These are **values identified less hypothetical costs, not profit estimates**, and the amount model ranks well but is not a calibrated dollar forecast. Scoring runs monthly on a laptop with no API fees; the outreach itself is the cost.

What we are not claiming. No causal effect: nobody was randomly assigned a call, so these are dollars *identified*, not *caused*; outreach impact remains unknown. The thank-you packet does not improve dollar ranking in our sensitivity run, and it goes mostly to donors whose project got funded, so its effect cannot be separated from the project succeeding. Transfer from a marketplace to a relational small nonprofit is a hypothesis. The \$25 cost is a placeholder for a figure from real practice.

Every number reproduces with python `evals/run_all.py`; sources in `evals/results/`.